Folder & Markdown Structure per Jake Van Clief

Interpretable Context Methodology (ICM) — what it prescribes, and how it maps onto the Drive project template. Compiled 2026-10-13 from the ICM paper and the reference repository.

1. Core idea

The folder structure **is** the agent architecture. Instead of a framework that orchestrates agents, a numbered set of folders holds the work, plain markdown files tell Claude where it is and what to do, and each stage hands off through files a human can open and edit. Claude "reads down" the layers and stops once it has what the task needs.

2. The five context layers

Layer	File / location	Question it answers	Size	Load
0	CLAUDE.md (workspace root)	Where am I?	~800 tokens	Always
1	CONTEXT.md (workspace root)	Where do I go?	~300 tokens	On entry
2	stages/NN-name/CONTEXT.md	What do I do?	200–500 tokens	Per task
3	references/, _config/, shared/, skills/	What rules apply?	500–2k tokens	Selectively; stable across runs
4	output/ (working artifacts)	What am I working with?	Varies	Selectively; changes every run

Layers 0–2 are routing. Layer 3 is the "factory" (configured once, treated by Claude as constraints). Layer 4 is the "product" (new each run, treated as input). Typical context per stage: 2,000–8,000 tokens, versus 30,000–50,000 for one monolithic prompt.

3. Reference workspace layout

```
workspace/
  CLAUDE.md                 Layer 0
  CONTEXT.md                Layer 1
  stages/
    01-research/
      CONTEXT.md            Layer 2  (stage contract)
      references/           Layer 3
      output/               Layer 4
    02-script/
      CONTEXT.md
      references/
      output/
    03-production/
      CONTEXT.md
      references/
      output/
  _config/                  Layer 3  (brand, voice, design system)
  shared/                   Layer 3  (cross-stage resources)
  skills/                   Layer 3  (bundled domain knowledge)
  setup/
    questionnaire.md        one-time onboarding
```

Stage names are numbered so order is visible in any file browser.

4. Stage contract (stage CONTEXT.md)

Every stage CONTEXT.md has three sections. The Inputs table is the main control point: it states exactly which files and which sections of them get loaded, so nothing enters context by accident.

```
# Stage 02 — Script

## Inputs
| Layer | Source                          | Section / scope | Why                |
|-------|---------------------------------|-----------------|--------------------|
| 3     | _config/voice.md                | "Tone rules"    | Match brand voice  |
| 4     | ../01-research/output/brief.md | whole file      | Source material    |

## Process
1. Draft the script from the brief.
2. Keep each script under 90 seconds.

## Outputs
| File                 | Location  |
|----------------------|-----------|
| script.md            | output/   |
```

5. Rules for writing the markdown files

Rule	Meaning
One stage, one job	A stage does not both research and write. Each reads a defined input and writes a defined output to its own folder.
CONTEXT.md under 80 lines	Routing files stay short. Reference files stay under 200 lines.
One-way references	If A points to B, B does not point back to A. No circular dependencies between stages.
Canonical sources	Each fact has one home. Other files point to it instead of copying it.
Selective section routing	Tables name the sections to load, not whole files.
Docs over outputs	Claude learns rules from reference docs, not from imitating previous outputs.
Declare every input	List each file and section in the Inputs table.
Concrete, checkable constraints	"Keep scripts under 90 seconds," not "keep scripts short."
Constraints vs. input	Persistent rules go in Layer 3. Per-run material goes in Layer 4.
Plain text interface	Markdown and JSON between stages, so any person or tool with a text editor can take part.
Every output is an edit surface	A human can open and edit any stage output before the next stage runs.
Routing file in big folders	A CONTEXT.md inside a large reference or config folder tells Claude which file to open.

6. Mapping to the Drive project template

Current template	ICM equivalent	Gap
Project Instructions (IDs + session rules)	Layer 0 (CLAUDE.md)	Matches. Keep it short.
_PROJECT_CONTEXT.md.txt	Layer 1 (workspace CONTEXT.md)	Matches. Works as the router plus project facts.
intake/_INTAKE_RULES.md.txt	Layer 2 (stage contract)	Has process steps but no Inputs / Outputs sections.
assets/_ASSETS_RULES.md.txt	Layer 3 (reference / source of truth)	Is a rules file, not yet a routed reference folder.
Files dropped in intake/	Layer 4 (working artifacts)	Matches.
(none)	stages/ with numbered folders	Template is flat: intake and assets only.

7. Suggested ICM-aligned template (an adaptation, not Van Clief's own)

The ICM sources describe multi-stage pipelines. A single-purpose Drive project needs only the parts below; add stage folders when a project has real sequential steps.

```
[ProjectName]/
  _PROJECT_CONTEXT.md.txt        Layer 1: purpose, routing, Active Files
  intake/
    _INTAKE_RULES.md.txt         Layer 2: Inputs / Process / Outputs
  assets/
    _ASSETS_RULES.md.txt         Layer 3: update-in-place rules
  stages/                        add only if the project has sequential steps
    01-name/
      CONTEXT.md
      references/
      output/
```

- Keep _PROJECT_CONTEXT.md.txt under 80 lines and move long reference material to separate files it points to.
- Rewrite _INTAKE_RULES.md.txt into Inputs / Process / Outputs sections.
- Have Project Instructions (Layer 0) say only where the folder is and which file to read first.

Sources

- Van Clief, Interpretable Context Methodology: Folder Structure as Agent Architecture — arxiv.org/pdf/2603.16021
- Reference repository — github.com/RinDig/Interpretable-Context-Methodology

Size limits and structural rules in section 5 come from the repository's contribution checklist. The paper itself gives no formal markdown style guide; its writing recommendations are inferred from its architecture and findings.